

Experiment Report

Atomicity

```
● Feb01 16:05 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 50000  
● Feb01 16:05 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 50000  
● Feb01 16:06 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 50000  
⦿ Feb01 16:06 zengweihang@Weihangs-Macbook hw3 $ go run .  
# github.com/weihang-z/CS6650-Scalable-System/hw3  
./atomic_counters.go:6:5: "sync/atomic" imported and not used  
● Feb01 16:08 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 26719  
● Feb01 16:08 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 23605  
● Feb01 16:08 zengweihang@Weihangs-Macbook hw3 $ go run .  
ops: 34834
```

Using `atomic.Uint64` and `ops.Add(1)` for each increment makes every update an atomic operation, eliminating race conditions and ensuring the result is stable and correct across runs (50,000). When we incremented `ops` using `ops++` without any synchronization, the program exhibited a classic data race: multiple goroutines read, modify, and write the same variable concurrently, causing many updates to be lost. As a result, repeated runs produced inconsistent and significantly smaller totals, indicating a nondeterministic behavior under concurrency.

Collections

```
⦿ Feb01 16:20 zengweihang@Weihangs-Macbook collections $ go run .  
fatal error: concurrent map writes  
^Csignal: interrupt
```

The program crashed with fatal error: concurrent map writes because Go's built-in map type is not safe for concurrent writes.

Mutex

```

• Feb01 16:24 zengweihang@Weihangs-Macbook mutex $ go run .
  total time = 20.256375ms
  len(m) = 50000
• Feb01 16:25 zengweihang@Weihangs-Macbook mutex $ go run .
  total time = 13.991291ms
  len(m) = 50000
• Feb01 16:25 zengweihang@Weihangs-Macbook mutex $ go run .
  total time = 11.494209ms
  len(m) = 50000
• Feb01 16:25 zengweihang@Weihangs-Macbook mutex $ go run .
  total time = 11.300375ms
  len(m) = 50000

```

We locked before every map write/read and unlocked immediately after and the program became stable and correct. In four runs, the total times were 20.256 ms, 13.991 ms, 11.494 ms, and 11.300 ms, giving a mean of about 14.261 ms. The key lesson is that a mutex provides correctness and safety for shared mutable data structures like Go maps, but it does so by serializing access to the critical section, and the mutex can become a throughput bottleneck.

RWMutex

```

• Feb01 16:27 zengweihang@Weihangs-Macbook rwmutex $ go run .
  total time = 20.087791ms
  len(m) = 50000
• Feb01 16:28 zengweihang@Weihangs-Macbook rwmutex $ go run .
  total time = 18.121ms
  len(m) = 50000
• Feb01 16:28 zengweihang@Weihangs-Macbook rwmutex $ go run .
  total time = 11.016667ms
  len(m) = 50000
• Feb01 16:28 zengweihang@Weihangs-Macbook rwmutex $ go run .
  total time = 11.142292ms
  len(m) = 50000
• Feb01 16:28 zengweihang@Weihangs-Macbook rwmutex $ go run .
  total time = 12.019208ms
  len(m) = 50000

```

The result was still correct and stable across runs: `len(m) = 50000` every time. The measured total runtimes were 20.088 ms, 18.121 ms, 11.017 ms, 11.142 ms, and 12.019 ms, with a mean of about 14.48 ms, which is very close to the plain Mutex version. This is expected because our workload is write-heavy, and an RWMutex must take an exclusive lock for writes, so it does not enable additional parallelism here.

Sync.Map

```
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 6.976708ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 5.581959ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 3.817334ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 3.3485ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 3.622625ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 3.477417ms  
len(m) = 50000  
• Feb01 16:34 zengweihang@Weihangs-Macbook sync.map $ go run .  
total time = 3.336083ms  
len(m) = 50000
```

Compared to our mutex-based wrappers, sync.Map was significantly faster, showing that the standard “map + external lock per operation” approach can introduce substantial contention/overhead. sync.Map’s built-in concurrency control can reduce that overhead for some workloads. The lesson: use a concurrency-safe structure when sharing maps across goroutines, and recognize that the “best” option depends on the read/write mix and contention pattern.

File Access

```
• Feb01 16:56 zengweihang@Weihangs-Macbook file_access $ go run .  
unbuffered: 209.5405ms  
buffered: 2.742708ms
```

```
• Feb01 16:58 zengweihang@Weihangs-Macbook file_access $ go run .  
unbuffered: 171.294041ms  
buffered: 2.711125ms
```

```
Feb01 16:58 zengweihang@Weihangs-Macbook file_access $ go run .
unbuffered: 158.02575ms
buffered:    5.012208ms
```

There is a 51× speedup on average for buffered output. The reason is that unbuffered mode performs a *huge number of small write operations*. `bufio.Writer` batches many small writes into fewer large writes, dramatically reducing syscall count and letting the OS handle larger, more efficient chunks.

Unbuffered can feel “safer” because each call pushes data to the OS immediately. However, for buffered, you must `Flush()` (and handle errors), and data can be lost if the program crashes before flushing/closing. It also uses a small amount of extra memory.

Context Switching

```
Feb01 17:03 zengweihang@Weihangs-Macbook context_switching $ go run .
GOMAXPROCS=1
total: 213.520375ms
avg per hand-off: 106ns

GOMAXPROCS=10
total: 258.297209ms
avg per hand-off: 129ns
Feb01 17:54 zengweihang@Weihangs-Macbook context_switching $ go run .
GOMAXPROCS=1
total: 227.312334ms
avg per hand-off: 113ns

GOMAXPROCS=10
total: 257.934542ms
avg per hand-off: 128ns
Feb01 17:54 zengweihang@Weihangs-Macbook context_switching $ go run .
GOMAXPROCS=1
total: 211.395542ms
avg per hand-off: 105ns

GOMAXPROCS=10
total: 258.352041ms
avg per hand-off: 129ns
```

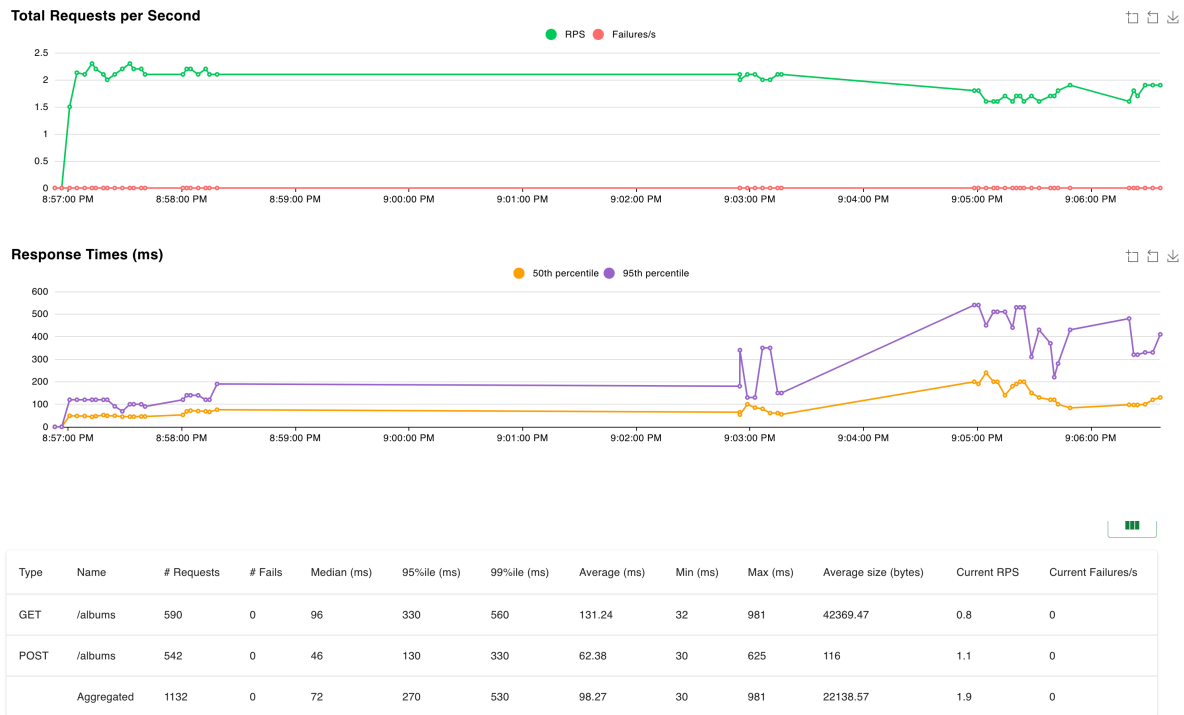
GOMAXPROCS=1 is faster with about 19% lower latency.

With one OS thread, Go avoids OS-level context switching entirely. It avoids OS thread scheduling, cache invalidation, and cross-core synchronization. In this ping-pong experiment, parallelism does not help, because the unbuffered channel introduces a *strict synchronization barrier* on every step. The two goroutines are forced to run sequentially, not in parallel, even on multiple cores.

Goroutine switches are *hundreds to thousands of times cheaper*. Containers don't change kernel scheduling cost. VMs add even more overhead. That's why Go can

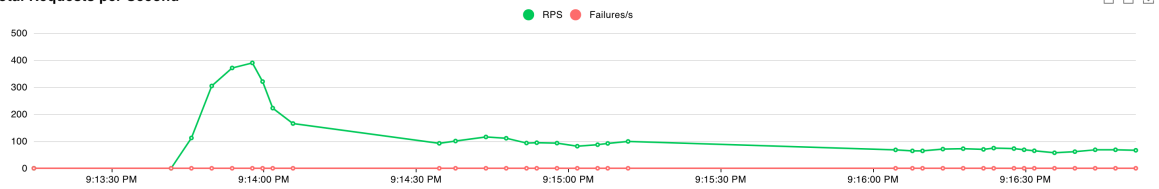
use huge numbers of goroutines efficiently.

Locust

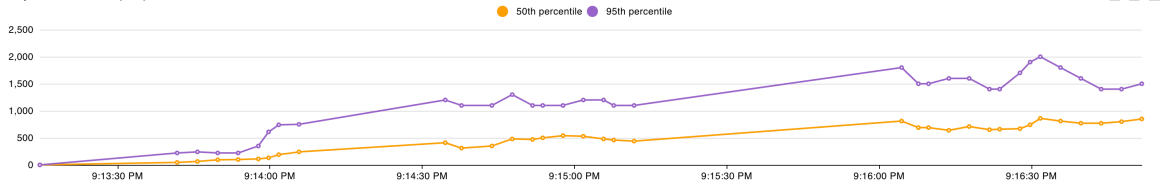


Using Locust with one worker and one simulated user, I tested GET and POST endpoints. The system sustained 2 requests/sec with zero failures, median latency under 100 ms, and p95 latency under 300 ms. Median for GET is 96ms, and median for POST is 46 ms. GET is slower than POST because it returns a larger payload, which may take more time.

Total Requests per Second

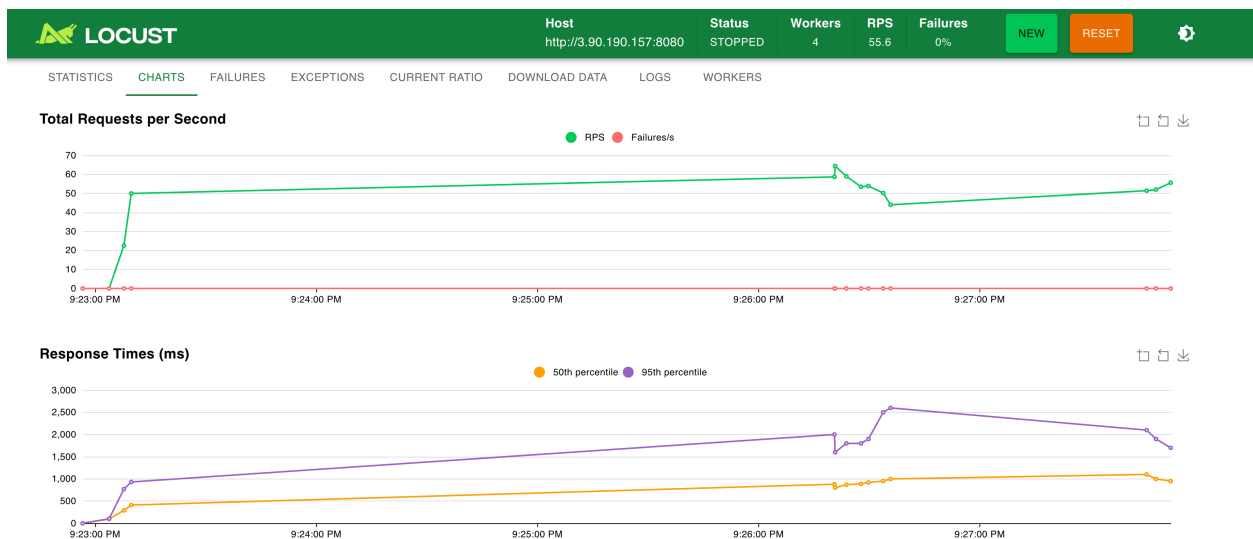


Response Times (ms)



Type	Name	# Requests	# Fails	Median (ms)	95thile (ms)	99thile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	15478	0	470	1300	2100	554.02	33	5330	449108.47	48.8	0
POST	/albums	5156	0	120	360	550	148.02	36	859	116	17.8	0
Aggregated		20634	0	280	1200	2000	452.57	33	5330	336914.75	66.6	0

50 concurrent users with a 3:1 GET/POST ratio, the system maintained zero failures but experienced significant tail latency. The latency increased significantly over time, especially for GET /albums, indicating a performance bottleneck likely due to large response payloads and a growing dataset during the test. The GET endpoint returned a large payload, and POST requests increased the dataset size during the run, which likely caused the response size and tail latency to grow.



LOCUST

Host

http://3.90.190.157:8080

Status

STOPPED

Workers

4

RPS

55.6

Failures

0%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

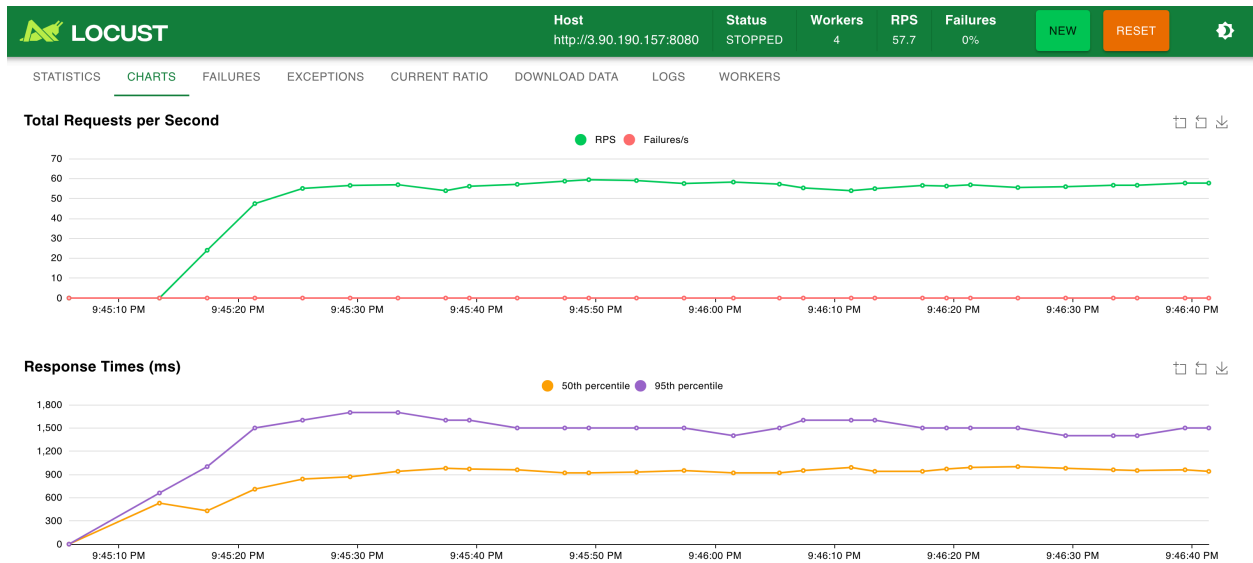
DOWNLOAD DATA

LOGS

WORKERS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	13262	0	910	1900	2600	1016.38	94	4814	1134803.9	40.5	0
POST	/albums	4451	0	150	390	570	179.79	34	958	116	15.1	0
	Aggregated	17713	0	760	1800	2500	806.16	34	4814	849674.57	55.6	0

Increasing the worker count from 1 to 4 did not improve throughput under the 50-user, 3:1 GET/POST workload. In fact, the latency increased, especially for GET /albums. A key factor is that the GET response payload grew substantially between runs (average size ~0.45 MB → ~1.13 MB), because POST requests increased the dataset size. Overall, the results are consistent with Amdahl's Law. Adding more load-generation workers does not necessarily bring linear scaling.



LOCUST

Host

http://3.90.190.157:8080

Status

STOPPED

Workers

4

RPS

57.7

Failures

0%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

WORKERS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	3693	0	1000	1600	1900	1071.56	250	2673	1536376.08	42.6	0
POST	/albums	1193	0	230	510	650	247.58	35	778	116	15.1	0
	Aggregated	4886	0	940	1500	1800	870.37	35	2673	1161272.05	57.7	0

After switching to FastHttps, RPS increased slightly (55.6 → 57.7), p95/p99 improved, but median/average worsened, and response size increased. FastHttpUser reduces client-side overhead and smooths tail latency, but the dominant bottleneck appears to be server-side serialization and network I/O for large GET responses.